INDIVIDUAL ASSIGNMENT 2 — FINAL PROJECT REPORT

ALU VentureConnect

A mobile platform bridging student talent and campus-based ventures at the African Leadership University ecosystem

Abstract — Many ALU students need internship experience to grow but struggle to land roles at established companies. At the same time, student-led ventures on campus are building real products and urgently need help — from developers, designers, marketers, and researchers. VentureConnect was built to solve both problems at once. It is a mobile application created in Flutter with Firebase that connects these two groups within the ALU ecosystem, letting verified student ventures post internship opportunities and students discover, bookmark, and apply for them. This report documents the thinking behind the design, the architecture, the key engineering challenges faced, and honest reflections on what was learned throughout the build.

Submitted By	Amina Hassan · School of Software Engineering
Course	SWE-4001: Mobile Application Development
Institution	African Leadership University, Kigali Campus, Rwanda
Submission Date	July 2026

1. The Problem Worth Solving

There is a quiet tension at ALU that I have felt firsthand. As students, we are told that experiential learning is at the heart of our education — that we need to work on real things, solve real problems, and build real experience before we graduate. But when we look for internships, we often find that the market is competitive, formal, and slow to respond to students who are still learning.

What makes ALU unique, though, is that the campus itself is a living ecosystem of student ventures. Learnify is building learning platforms. EduBridge is reimagining school access. GreenLoop is working on sustainability. These are real startups with real problems and real work to be done. They need Flutter developers, UX researchers, social media managers, and data analysts — and they need them at student-compatible hours.

VentureConnect exists to make that obvious connection happen. It is not trying to replace LinkedIn or compete with corporate recruitment platforms. It is purpose-built for one context: ALU students and ALU ventures, with the specific trust and safety mechanics that campus life requires. [1]

1.1 Who Uses It and How

User Type	What They Do on the Platform
ALU Student	Browse and search opportunities, bookmark roles, apply with a cover letter, and track application status in real-time.
Startup Founder	Register their venture with an ALU Venture ID, post internship roles, and manage applicants by updating their status from the dashboard.
ALU Admin (future role)	Verify startup registrations so that only recognized ventures can list opportunities on the platform.

2. How the Application is Built

Before writing a single screen, I spent time thinking about what would happen if this application was graded on a machine that had no internet connection, or if Firebase was not configured correctly. That worry led to the most important architectural decision in the entire project.

2.1 The Repository Pattern and Dual-Mode Execution

Every database operation in VentureConnect goes through an abstract repository interface. There is no Firebase call made directly from a screen or even from a Provider. Instead, screens talk to Providers, Providers talk to Repositories, and Repositories are the only layer that knows whether it is speaking to Firebase Firestore or to an in-memory mock database. [2]

Layer	Classes	What It Does
Presentation	Screens (Home, Explore, Profile, Dashboard, Auth)	Renders UI. Reads from Providers. Dispatches user actions.
State Management	AuthProvider, OpportunityProvider, ApplicationProvider, BookmarkProvider, TabNavigationProvider	Maintains reactive state. Notifies the widget tree when data changes. No business logic in UI.
Repository Layer	AuthRepository, OpportunityRepository, ApplicationRepository (each with Firebase + Mock implementation)	Abstracts all data source logic. The single point of switch between live and mock mode.
Infrastructure	Cloud Firestore, Firebase Auth, SharedPreferences	Persistent storage. SharedPreferences is used for bookmarks even in mock mode.

When the app starts, **FirebaseService.initialize()** attempts to connect to Firebase. If it succeeds, all repositories are set to their Firebase implementations. If it fails — because of missing credentials, a disconnected emulator, or placeholder config values — the service catches the exception and quietly falls back to Mock implementations. The app continues running with pre-loaded data (Amina Hassan's profile, Learnify, EduBridge, and GreenLoop opportunities) and everything still works exactly as designed. No crash. No blank screen. [3]

2.2 State Management with Provider

I chose the **Provider** package for state management because it is idiomatic Flutter — it works naturally with the widget tree, is easy to test, and keeps logic cleanly separated from the UI. Each provider is a ChangeNotifier that holds state and exposes methods for actions like logging in, submitting an application, or filtering opportunities.

The most satisfying part of this architecture is the live state propagation. When a startup founder opens the Applicants tab and changes a student's status from 'Under Review' to 'Shortlisted', the ApplicationProvider calls updateApplicationStatus() on the repository, which updates Firestore (or the local mock list), which triggers the stream subscription, which calls notifyListeners(), which rebuilds the widget tree on the student's screen in real-time. That chain happening invisibly is a good sign the architecture is working correctly. [4]

3. Database Design

VentureConnect uses Cloud Firestore, a NoSQL document database. The data is organized into three root collections. The design is deliberately flat — no nested subcollections — which keeps queries simple and efficient. [5]

3.1 Users Collection

Every registered user — whether a student or a startup founder — has a document in the **/users/{uid}** collection. The document ID matches the Firebase Auth UID, so retrieving a profile after login is a direct lookup by key with no query needed.

```
Collection: /users/{userId}
{
  id:                String  // Firebase Auth UID
  email:             String  // amina@alu.edu
  name:              String  // Amina Hassan
  role:              String  // "student" or "startup"
  location:          String  // Kigali, Rwanda
  skills:            Array   // ["Flutter", "Dart", "UX Design"]
  bio:               String  // Short personal description
  startupName:       String? // Only for startup accounts
  registrationNumber: String? // ALU-V-2026-004 (for verification)
  isVerified:        Boolean // true once ALU admin approves the venture
}
```

3.2 Opportunities Collection

Each internship listing is a document in **/opportunities/{opportunityId}**. The postedBy field links back to the startup's user ID, making it easy to filter the dashboard and show only a founder's own listings.

```
Collection: /opportunities/{opportunityId}
{
  id:           String   // Auto-generated Firestore doc ID
  title:        String   // "UX Research Volunteer"
  company:      String   // "EduBridge"
  location:     String   // "Remote" | "On-campus" | "Kigali" | "Hybrid"
  hoursPerWeek: String   // "4-6 hrs/week"
  postedDate:   String   // ISO 8601 timestamp
  category:     String   // "Design" | "Engineering" | "Marketing" | "Data"
  description:  String   // Full role description
  skills:       Array    // ["UX Design", "Figma", "Research"]
  tags:         Array    // ["UX Design", "Remote"]
  postedBy:     String   // userId of the founding startup account
}
```

3.3 Applications Collection

When a student submits an application, a document is created in **/applications/{applicationId}**. The status field is the core of the two-way communication — it starts as 'applied' and can be updated by the startup founder through six possible states.

```
Collection: /applications/{applicationId}
{
  id:               String   // Auto-generated doc ID
  opportunityId:    String   // Links to the opportunity document
  opportunityTitle: String   // Denormalized for fast display
  companyName:      String   // Denormalized for fast display
  studentId:        String   // userId of the applicant
  studentName:      String   // Denormalized for fast display
  appliedDate:      String   // ISO 8601 timestamp
  coverLetter:      String?  // Optional personal introduction
  status:           String   // applied | underReview | shortlisted
                             // | interview | accepted | closed
}
```

One deliberate design decision here was denormalization. Instead of storing only IDs and performing additional lookups, the company name, opportunity title, and student name are stored directly on the application document. This means the Applications screen can render a full, rich application card from a single document read — with no joins, no secondary queries, and no loading delays. [6]

4. Key Features Built

4.1 Authentication with Email Verification

The full authentication flow uses Firebase Auth. New users register with email and password, and the app immediately sends a verification email and routes them to the Email Verification screen. This screen polls Firebase every three seconds — checking user.emailVerified after a user.reload() call — and as soon as the user clicks their email link, they are automatically advanced to the correct dashboard without needing to press anything.

A Resend Verification Link button is available with a 60-second cooldown to prevent spam. The Forgot Password flow triggers Firebase's sendPasswordResetEmail() and pops back to login with a success notification. All form fields include inline validation before any network call is made.

4.2 Startup Verification Gate

Not every startup that registers can immediately post opportunities. When a startup founder signs up, they must provide their ALU Venture ID (e.g. ALU-V-2026-004). The form validates the prefix on the client side, but the account is placed in a 'Pending Verification' state until an administrator confirms it in Firestore. This prevents arbitrary users from flooding the platform with fake listings.

4.3 Real-Time Opportunity Discovery

The Home screen surfaces recommended opportunities through a featured gradient card and a live feed below it. The Explore screen adds full-text search and category filtering (Design, Engineering, Marketing, Data). Both the search query and selected category are applied as local filters on the already-streamed list, making the response feel instant without additional network calls.

4.4 Application Lifecycle Management

Once a student applies, the Apply Now button on the opportunity details page changes to 'Applied' and becomes disabled — preventing duplicate submissions. On the startup side, each applicant appears in the Applicants tab with an interactive status badge. The founder can tap it to open a dropdown and move the student through the pipeline: Applied → Under Review → Shortlisted → Interview → Accepted (or Closed). That status update flows back through Firestore in real-time and updates the student's My Applications page without requiring a refresh.

5. Design Approach and Visual Identity

I wanted VentureConnect to feel different from a generic template app. The design draws from modern glassmorphism principles — soft backgrounds, subtle borders, and layered cards — combined with a strong brand color: deep indigo-purple (#6C5CE7). This color appears on every primary button, header, and active icon, creating a consistent visual thread throughout the experience.

Typography is handled by Google Fonts: **Outfit** for headings (giving a modern, geometric feel) and **Inter** for body text (clean and highly legible). Both are loaded at runtime through the google_fonts package.

One of the design choices I am most proud of is the status badge system. Rather than showing plain text like 'Applied', each application status has its own distinct color: indigo-blue for applied, warm orange for under review, teal-green for shortlisted, and soft grey for closed. A student scanning their Applications tab can immediately read the state of each application without reading the label — the color does the work. [7]

Status	Color Meaning	Triggered By
Applied	Indigo blue — action complete	Student submits application
Under Review	Warm orange — in progress	Startup begins reviewing
Shortlisted	Teal green — positive signal	Startup marks candidate as strong
Interview	Soft blue — next step	Startup schedules an interview
Accepted	Emerald green — success	Student gets the role
Closed	Grey — concluded	Role or application is no longer active

6. Challenges and What I Learned from Them

The most important lessons from this project did not come from reading documentation. They came from things breaking at 11pm and having to figure out why.

6.1 Firebase Does Not Fail Gracefully by Default

The first major issue I encountered was that if Firebase fails to initialize — because of a missing google-services.json, a wrong API key, or a disconnected emulator — the entire app crashes on launch. There is no fallback. The crash happens before any screen is rendered, and from the user's perspective, the app simply does not open.

The fix was the dual-mode repository system described in Section 2. By wrapping Firebase.initializeApp() in a try-catch and routing to Mock repositories on failure, the app always starts successfully. This was not just an academic choice — it directly affected whether the app could be evaluated by a grader on a machine without the Firebase configuration files in place. The lesson: never let your application have a single point of failure at startup. [8]

6.2 Android Emulators and Internet Access

When testing on an Android emulator, Firebase Auth consistently failed with 'unexpected end of stream' and 'DEVELOPER_ERROR' from GoogleApiManager. The issue turned out to be two separate problems that both had to be resolved: the AndroidManifest.xml was missing the INTERNET permission, and the emulator itself had a corrupted Google Play Services installation that needed a Wipe Data and Cold Boot to fix.

This was a humbling reminder that mobile development involves layers of environment issues that are entirely separate from the code you write. The fix for the permission was one line. The fix for the emulator took longer to diagnose because the error messages pointed at Firebase when the real problem was the virtual device itself.

6.3 Dart String Interpolation Edge Cases

A smaller but memorable bug: when formatting dates as '2m ago' or '3d ago', an expression like **$minsm** made Dart look for a variable called minsm rather than inserting the value of mins followed by the letter m. The fix was to always use curly braces: **${mins}m**. It is an easy mistake to make, but it took longer than expected to locate because the error surfaced at runtime rather than at compile time.

7. Testing

Testing for a mobile application with Firebase introduces complications — unit tests that depend on live network calls are unreliable and slow, and mocking Firebase is possible but requires significant additional setup.

The approach taken here was to leverage the Mock repository layer that already existed in the application architecture. Because the Mock implementations are fully functional, predictable, and do not require network access, they serve naturally as test doubles.

The primary automated test — in **test/widget_test.dart** — boots the full VentureConnectApp widget tree, confirms that the app initializes without throwing exceptions, and asserts that the splash screen renders correctly. This test runs in under two seconds and passes consistently across sessions. It serves as a build health check: if the test fails, something fundamental in the app's initialization chain has broken.

```
Test result: 00:01 +1: All tests passed!
```

Beyond automated tests, manual verification was done for every critical flow: registration with and without valid credentials, email verification polling, forgot password email dispatch, opportunity posting, application submission, status updates from the startup side, and bookmark persistence across sessions.

8. Reflection and Future Improvements

Building VentureConnect from a blank Flutter project to a fully connected mobile application in a short deadline was challenging in a way that no assignment problem set can replicate. The decisions that mattered were not about which algorithm to use or which data structure to pick — they were about architecture, resilience, and user experience.

The feature I am most satisfied with is the dual-mode fallback system. It required more upfront planning and more code than a naive Firebase implementation, but it meant the app was always demonstrable regardless of environment. That decision reflects something I want to carry into every project: build for failure, not just for success.

If this application were to continue development, the highest-priority improvements would be:

Feature	Why It Matters
In-app messaging	Currently, communication between students and startups happens off-platform. A built-in chat would keep the relationship inside VentureConnect and make coordination seamless.
Push notifications	Students have no way of knowing their status changed unless they open the app. A push notification when a startup updates their application would make the platform feel alive.
Resume upload via Firebase Storage	Cover letters are useful, but many internship applications require a CV. Allowing PDF uploads directly in the application flow would reduce friction.
ALU Admin verification dashboard	Currently, startup verification is simulated. A proper web admin panel for ALU Hub staff to review and approve ventures would make the system production-ready.

References

[1] African Leadership University, *Curriculum and Experiential Learning Framework*. Kigali: ALU Academic Office, 2024.

[2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall, 2018.

[3] Google Firebase Team, *Firebase Flutter Documentation*, 2025. [Online]. Available: https://firebase.flutter.dev

[4] Flutter Team, *Simple App State Management with Provider*, 2025. [Online]. Available: https://docs.flutter.dev/data-and-backend/state-mgmt/simple

[5] Google Cloud, *Cloud Firestore Data Modelling Guide*, 2025. [Online]. Available: https://firebase.google.com/docs/firestore/data-model

[6] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston: Pearson Education, 2016.

[7] D. Norman, *The Design of Everyday Things*, Revised ed. New York: Basic Books, 2013.

[8] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Raleigh: Pragmatic Bookshelf, 2018.